

Notes about this example:

Firstly, this example is for validating the multi-scaling reservoir computing framework does not need the assumption about the deterministic part F as proposed in the Lim et al. 2020. (Algorithm III.1. in their paper [1]), below are their algorithm:

(S1) **Feature extraction*. Extract the fast driving signal using the data for x and the known expression for F ,*

$$f(t_i) = \frac{x(t_{i+1}) - x(t_i)}{\Delta t} - F(x(t_i), t_i)$$

for $i = 0, 1, \dots, N - 1$, where Δt is a uniform time step.

(S2) *Machine learning. Using $(f(t_i))_{i=0, \dots, N-1}$ as the training data, predict the values of f for $M + 1$ time steps into the future (i.e., beyond time $t_{obs} - \Delta t$) using a supervised learning algorithm (for instance, Algorithm III.2 in Sec. IV) that is best fit for the task to infer $(f(t_N), f(t_{N+1}), \dots, f(t_{N+M}))$.*

(S3) *Numerical integration. Numerically evolve the system up to time t_{N+M} , with $(f(t))_{t=t_0, \dots, t_{N+M}}$ in place of $\frac{\sigma}{\epsilon} G(\xi(t))$ in (2), using the step size Δt . The predicted values for x in the time window $[t_{obs} + \Delta t, t_f = (N + M)\Delta t]$ are then obtained from the resulting numerical solutions.*

Due to the inherent randomness in the matrices of RC as noted in [2], "Using random matrices in an RC presents problems: many perform well, but others do not all". Thus, even with a fixed set of hyperparameters, different matrices produce differing results. Accordingly, the authors (Lim et al.) performed 50 noise predictions with a fixed hyperparameter set, utilizing data prior to t_{obs} (as described in the caption of their Fig. 2, only a segment before $t = 37 < \tau_0 = 38.52$, at most $N + 1 = 3701$ data point was available for training).

To adhere to their protocol, we executed 50 predictions using the noise separated before t_{obs} , iteratively adding it into our model at each time step to simulate a dynamical system. For these colored noise examples, we provide the 50 noise prediction results and deterministic matrices as npy files. This facilitates reproducibility across different computers, given that numpy random seeds and matrices can be influenced by device disparities (e.g., Intel vs. AMD architectures, Windows, Linux) that differences may substantially accumulate in our iterative dynamical system.

[1] Lim, S. H., Theo Giorgini, L., Moon, W., & Wettlaufer, J. S. (2020). Predicting critical transitions in multiscale dynamical systems using reservoir computing.

Chaos: An Interdisciplinary Journal of Nonlinear Science, 30(12).

[2] Gauthier, D. J., Bollt, E., Griffith, A., & Barbosa, W. A. (2021). Next generation reservoir computing. *Nature communications*, 12(1), 1-8.

A note about the separated noise:

In our train2 function, we have:

```
In [ ]: for i in range(self.dimensions):

        X=self.dataset[index][np.array(range(int(self.P/self.dimensions)))
        Y=self.S[np.array(range(int(self.P/self.dimensions)))*self.dimensions]
        SampledNoise=[]
        for j in range(X.shape[1]):
            XX=np.array(X[:,j])
            YY=np.array(Y[:,j])
            NoiseTemp=XX-YY
            Bins=max(5,int(np.sqrt(X.shape[0])))
            hist, bins = np.histogram(NoiseTemp, bins=Bins,range=(min(NoiseTemp),max(NoiseTemp)))
            bin_midpoints = bins[:-1] + np.diff(bins)/2
            cdf = np.cumsum(hist)
            cdf = cdf / cdf[-1]
            values = np.random.rand(X.shape[0])
            value_bins = np.searchsorted(cdf, values)
            random_from_cdf = bin_midpoints[value_bins]
            if j<0:#show the fitting to the noise distribution
                plt.subplot(121)
                plt.hist(NoiseTemp, Bins)
                plt.subplot(122)
                plt.hist(random_from_cdf, Bins)
                plt.show()

        SampledNoise.append(random_from_cdf)
        self.SampledNoise.append(SampledNoise)
```

In this function, when the noise distribution has only one sample in each dimension (i.e., $X.shape[0] = 1$), `random_from_cdf` is identically equal to `NoiseTemp` due to the histogram collapsing into a single bin. Specifically, `np.histogram` auto-expands the bin range symmetrically around the lone data point, but with only one value, the density concentrates in the central bin, and the CDF-based sampling consistently returns that bin's midpoint—exactly `NoiseTemp`. Therefore, the `random_from_cdf` sampling code is redundant and can be removed, as demonstrated below.

```
In [16]: import numpy as np

# Simulate the data scenario: X.shape = (1, 4), i.e., 1 sample and 4 time steps
X = np.array([[10.0, 2e-5, 3.0, 12345]]) # Assume true values
Y = np.array([[0.9, 1e-4, 2.7, 11]]) # Assume predicted values

SampledNoise1 = []
SampledNoise2 = []

for j in range(X.shape[1]):
```

```

XX = X[:, j] # Take the predicted value at the j-th time step (shape=(1,))
YY = Y[:, j] # Take the true value at the j-th time step (shape=(1,))
NoiseTemp = XX - YY # Calculate the noise (shape=(1,))

print(f"\n--- Time Step {j} ---")
print("NoiseTemp:", NoiseTemp)
Bins = max(5, int(np.sqrt(X.shape[0]))) # Bins=5
hist, bins = np.histogram(NoiseTemp, bins=Bins, range=(min(NoiseTemp), max(NoiseTemp)))
bin_midpoints = bins[:-1] + np.diff(bins)/2
print("bin_midpoints", bin_midpoints)
cdf = np.cumsum(hist)
cdf = cdf / cdf[-1]
values = np.random.rand(X.shape[0])
value_bins = np.searchsorted(cdf, values)
print("value_bins", value_bins)
random_from_cdf = bin_midpoints[value_bins]
print("Random Samples:", random_from_cdf)
SampledNoise1.append(random_from_cdf)
SampledNoise2.append(NoiseTemp)

print("\nFinal SampledNoise1:", SampledNoise1)
print("\nFinal SampledNoise2:", SampledNoise2)

```

--- Time Step 0 ---

NoiseTemp: [9.1]

bin_midpoints [8.7 8.9 9.1 9.3 9.5]

value_bins [2]

Random Samples: [9.1]

--- Time Step 1 ---

NoiseTemp: [-8.e-05]

bin_midpoints [-4.0008e-01 -2.0008e-01 -8.0000e-05 1.9992e-01 3.9992e-01]

value_bins [2]

Random Samples: [-8.e-05]

--- Time Step 2 ---

NoiseTemp: [0.3]

bin_midpoints [-0.1 0.1 0.3 0.5 0.7]

value_bins [2]

Random Samples: [0.3]

--- Time Step 3 ---

NoiseTemp: [12334.]

bin_midpoints [12333.6 12333.8 12334. 12334.2 12334.4]

value_bins [2]

Random Samples: [12334.]

Final SampledNoise1: [array([9.1]), array([-8.e-05]), array([0.3]), array([12334.])]

Final SampledNoise2: [array([9.1]), array([-8.e-05]), array([0.3]), array([12334.])]